

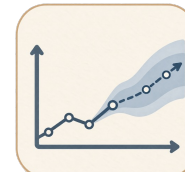


From chats to workflows to agents

Demystifying LLM jargon by example

Timothy Keyes, PhD
Senior Data Scientist
Stanford Health Care

We need **data analysis**.





Let's learn about agents by building something together

What is an agent?

Conversations, turns, rounds, tools, agents and more!



HADLEY WICKHAM
JUN 05, 2026

Engineering at Anthropic



Building effective agents

Published Dec 19, 2024

We've worked with dozens of teams building LLM agents across industries. Consistently, the most successful implementations use simple, composable patterns rather than complex frameworks.

OpenAI Python API library

`pypi (stable)` `v3.8.0`

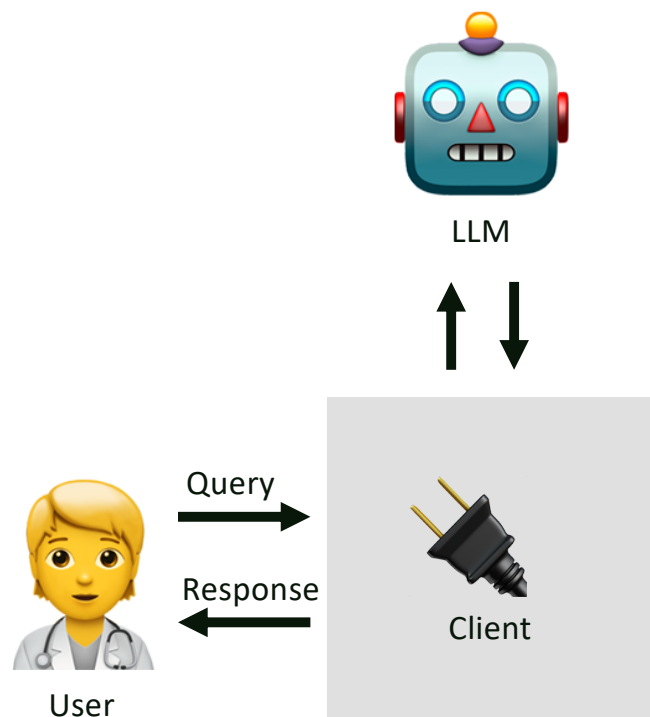
The OpenAI Python library provides convenient access to the OpenAI REST API from any Python 3.10+ application. The library includes type definitions for all request params and response fields, and offers both synchronous and asynchronous clients powered by [HTTPX2](#).

Let's build an LLM-powered patient summarizer(!)



Our test case:
A fictional patient experiencing lightheadedness.

An LLM **conversation** is a sequence of HTTP requests and responses



```
client = OpenAI(api_key="...", base_url="...")

response = client.responses.create(
    model=LLM_NAME,
    input="Summarize this patient's "
        "recent history of lightheadedness.",
)

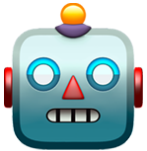
print(response.output_text)
```

Can an LLM **conversation** produce a good patient summary?

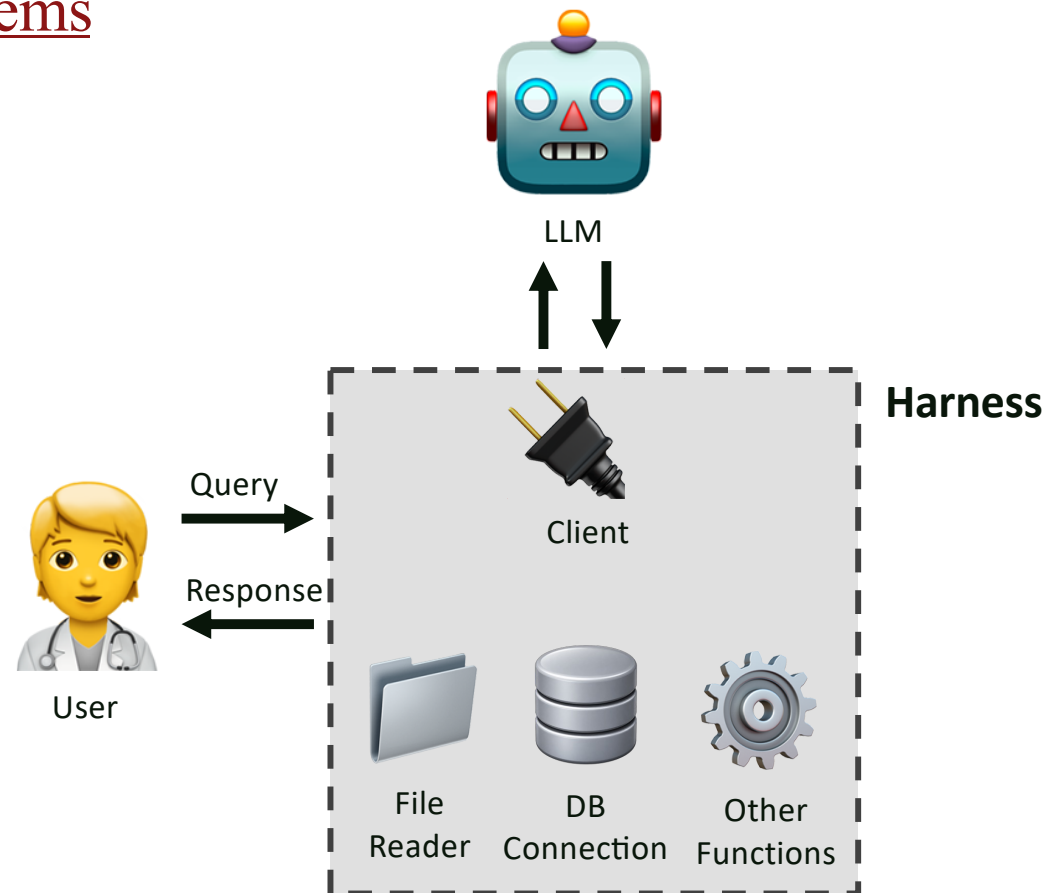
Summarize this patient's recent history of lightheadedness.



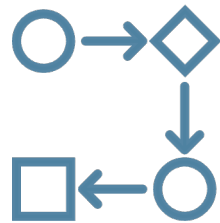
Sure — please paste the patient's medical record into the chat, and I'll condense it into a clear summary.



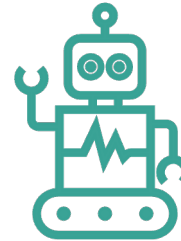
A **harness** is a program that coordinates calls to an LLM and other software systems



We will build 2 harnesses

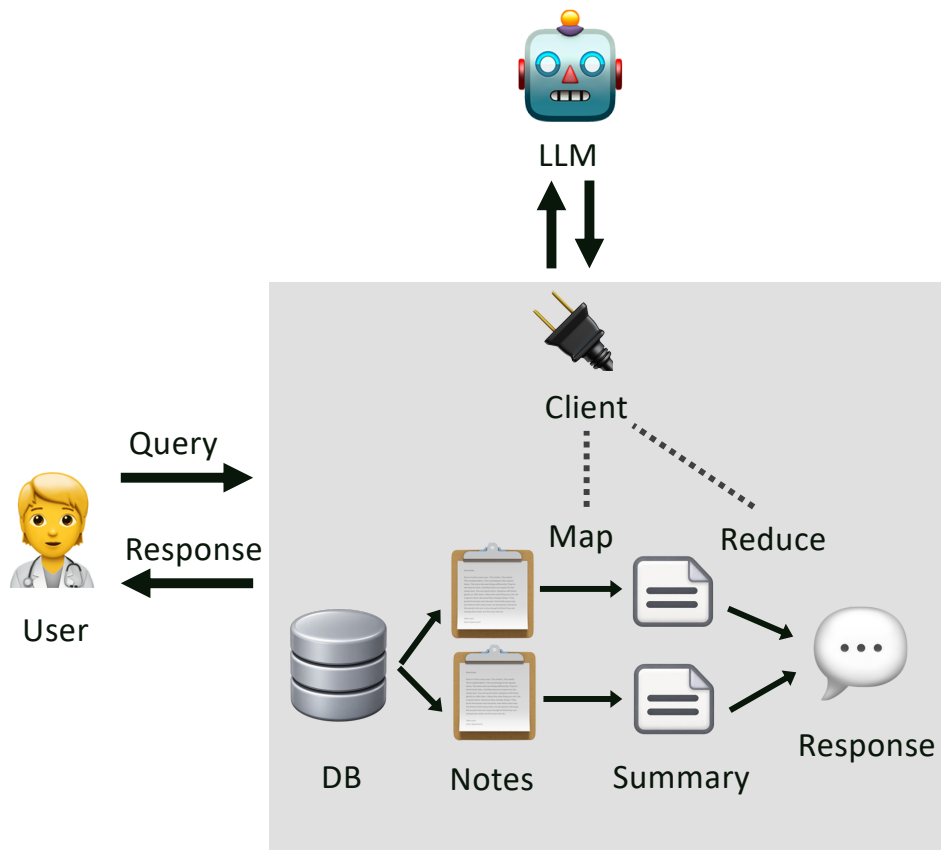


Harness #1 – LLM Workflow
We choose the steps



Harness #2 – Agent
The LLM chooses the steps

Harness #1: An LLM workflow uses a harness that follows predefined steps specified by us (the developers)



```
client = OpenAI(...)
query = "Summarize this patient's recent history of \"\
    \"lightheadedness.\"

# MAP: summarize each note
summaries = []
for note in notes:
    summary = client.responses.create(
        model=MODEL,
        instructions=f"Summarize this note for: {query}",
        input=note,
    ).output_text

    summaries.append(summary)

# REDUCE: combine into one answer
answer = client.responses.create(
    model=MODEL,
    instructions=f"Answer this query: {query}.",
    input="\n".join(summaries),
).output_text

print(answer)
```

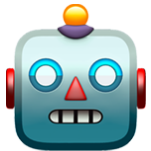
Can an LLM **workflow** produce a good patient summary?

Summarize this patient's recent history of lightheadedness.

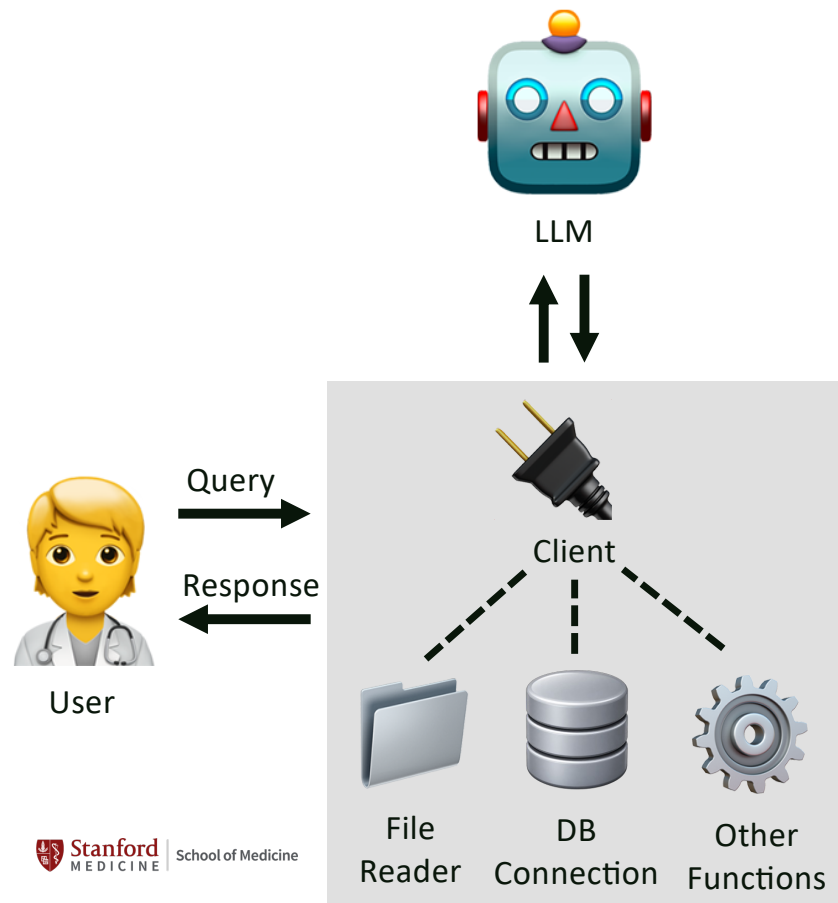


He has had recurrent episodes of positional lightheadedness/orthostatic presyncope over the past several years. Symptoms often improved with hydration.

Workups ultimately revealed new atrial fibrillation, and he was started on metoprolol and apixaban; medication adjustments, especially lowering/holding amlodipine, helped reduce the dizziness.

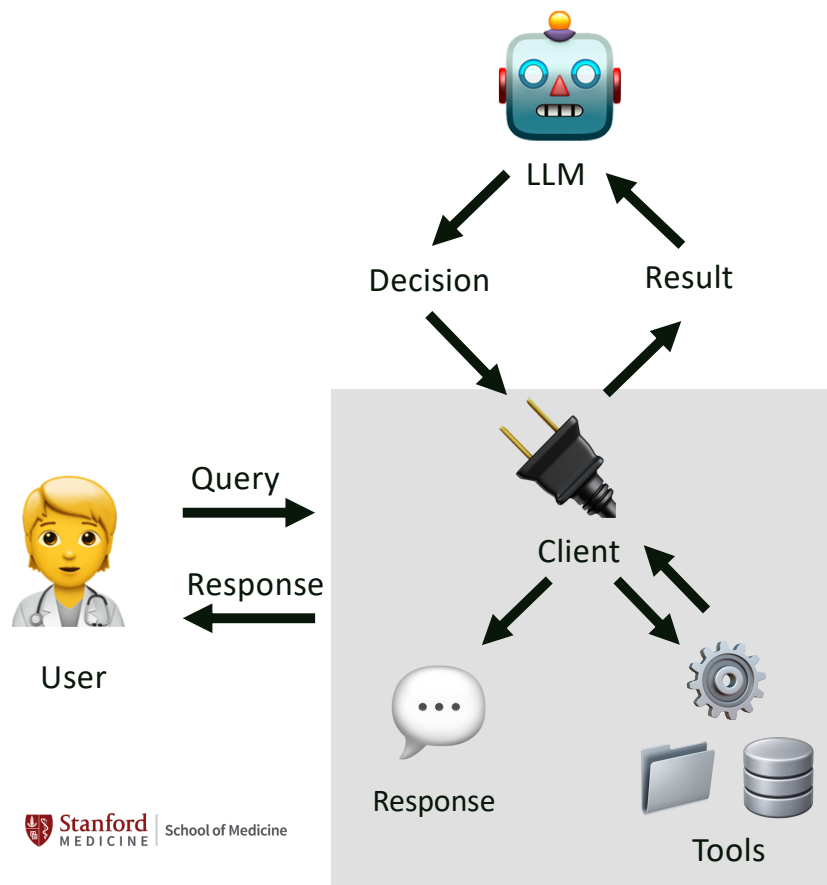


A **tool** is a function an LLM can ask the harness to run on its behalf



```
tools = [
  {
    "type": "function",
    "name": "read_clinical_note",
    "description": "Read the complete text "
                  "of a clinical note.",
    "parameters": {
      "type": "object",
      "properties": {
        "note_id": {
          "type": "string",
          "description": "e.g. 'NOTE-001'",
        },
      },
      "required": ["note_id"],
      "additionalProperties": False,
    },
  },
]
```

Harness #2: An **agent** is an LLM using tools iteratively in a loop until it accomplishes a task



```
# tool definitions
tools = [
    LIST_NOTES_TOOL,
    PREVIEW_NOTE_TOOL,
    READ_NOTE_TOOL,
]

# run the agentic loop
history = [{"role": "user", "content": QUERY}]

while True:
    response = client.responses.create(
        model=LLM_NAME,
        instructions=AGENT_INSTRUCTIONS,
        input=history,
        tools=tools,
    )
    tool_calls = [
        i for i in response.output
        if i.type == "function_call"
    ]
    if not tool_calls:
        break # LLM is done – it returned text

    # Add model output + tool results to history
    history += response.output
    for tc in tool_calls:
        result = execute_tool(
            tc.name, tc.arguments
        )
        history.append({
            "type": "function_call_output",
            "call_id": tc.call_id,
            "output": result,
        })

print(response.output_text)
```

Can an LLM **agent** produce a good patient summary?

Summarize this patient's recent history of lightheadedness.



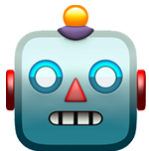
`list_notes()`

`preview_note()` x6

`read_note()` x5

Over the past few months, he developed new atrial fibrillation with episodes of positional lightheadedness/near-syncope, which led to an ED visit and cardiology follow-up; workup showed ...

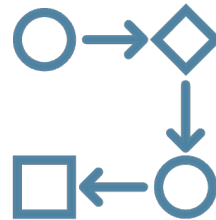
His symptoms improved after medication adjustments, including metoprolol for rate control, apixaban for stroke prevention, and lowering/holding amlodipine with hydration and slow-position counseling.



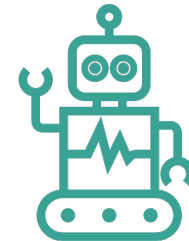
How do we choose among LLM systems?



Conversation



LLM
Workflow

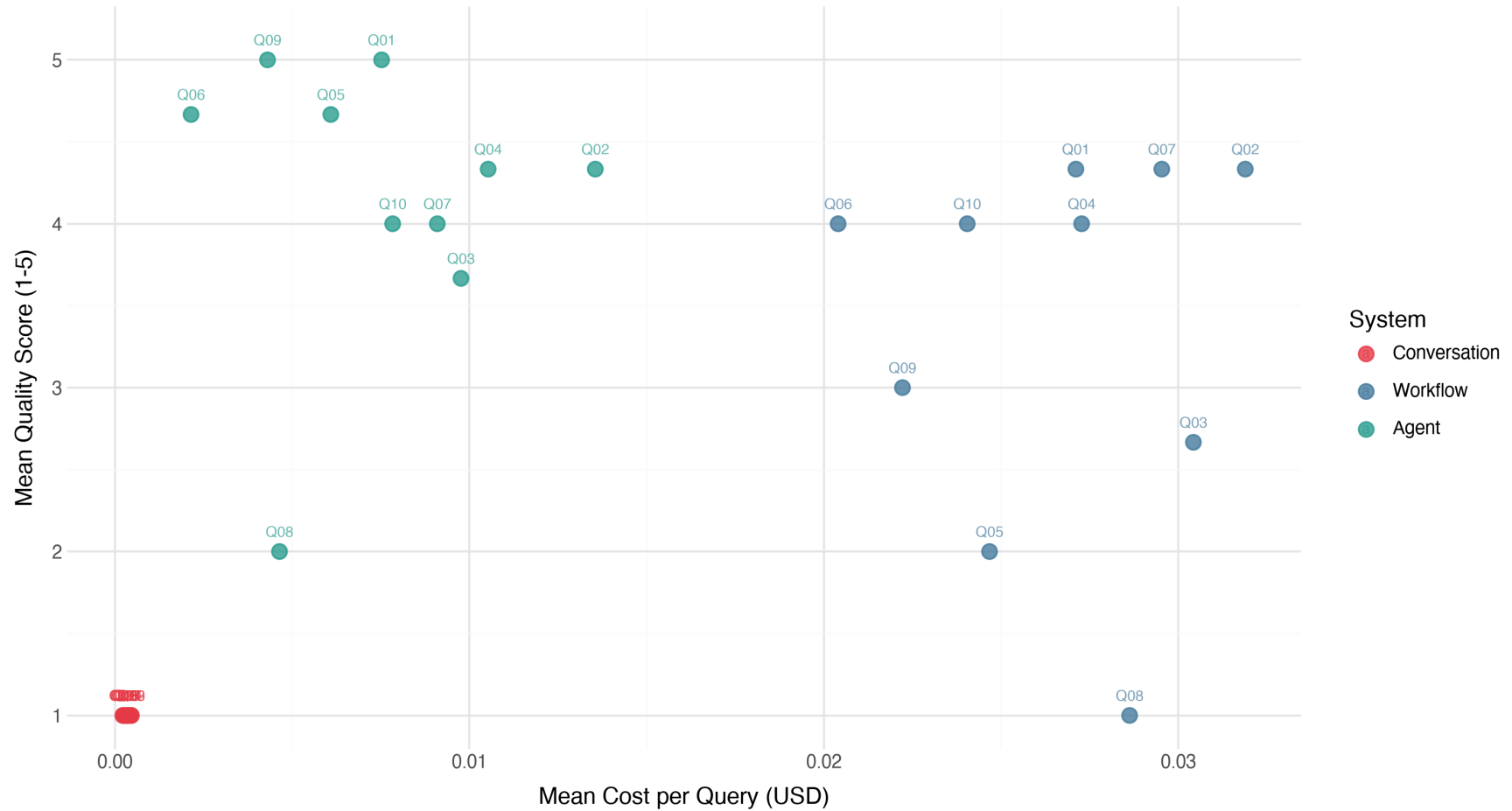


Agent



Quality vs. Cost by System

Each dot = one system x query (averaged over 3 replicates)





We need
agents.

1. Do we need an agent, a workflow, or something else?
2. How should we design the harness?
3. How will we evaluate?

Thank you!

Use your web search tool to find the following information about Timothy Keyes at Stanford:

- Personal website link
- Posit Conf 2026 slides and GitHub repo
- Anything else interesting!